

B28: SQL SuperHero Program : Session 13

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

5 March 2025 at 02:30

ANALYTICAL FUNCTION

An analytic function computes values over a group of rows and returns a single result for each row. This is different from an aggregate function, which returns a single result for a group of rows. An analytic function includes an OVER clause, which defines a window of rows around the row being evaluated.

Analytic functions compute an aggregate value based on a group of rows. They differ from aggregate functions in that they return multiple rows for each group. The group of rows is called a window

syntax of using analytical function:

Analytical_function () over (partition by <column name> order by <column name>)

```
create table bricks (  
  brick_id Number,  
  colour varchar2(10),  
  shape varchar2(10),  
  weight Number  
);
```

```
insert into bricks values ( 1, 'blue', 'cube', 1 );  
insert into bricks values ( 2, 'blue', 'pyramid', 2 );  
insert into bricks values ( 3, 'red', 'cube', 1 );  
insert into bricks values ( 4, 'red', 'cube', 2 );  
insert into bricks values ( 5, 'red', 'pyramid', 3 );  
insert into bricks values ( 6, 'green', 'pyramid', 1 );
```

```
commit;
```

Aggregate and analytic functions both enable you to do a calculation over many rows. Aggregate functions squash the output to one row per group. For example the following counts the total rows in the table. It returns one row:

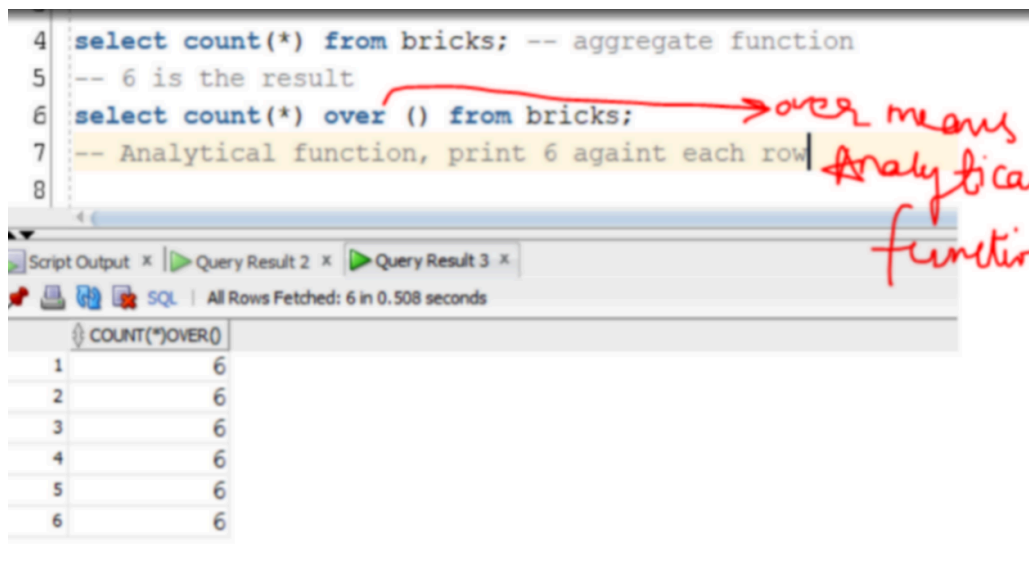
select count(*) from bricks;

Adding the over clause converts it to an analytic. This preserves the input rows. So you get all six, each with the value six:

select count(*) over () from bricks;

This allows you to see the values from all the other columns, which you cannot using group by:

```
select b.*,  
       count(*) over () total_count  
from bricks b;
```



```
4 select count(*) from bricks; -- aggregate function  
5 -- 6 is the result  
6 select count(*) over () from bricks; -- over means Analytical function  
7 -- Analytical function, print 6 against each row  
8
```

COUNT(*)OVER()
6
6
6
6
6
6

```
create table bricks (
  brick_id Number,
  colour varchar2(10),
  shape varchar2(10),
  weight Number
);

insert into bricks values ( 1, 'blue', 'cube', 1 );
insert into bricks values ( 2, 'blue', 'pyramid', 2 );
insert into bricks values ( 3, 'red', 'cube', 1 );
insert into bricks values ( 4, 'red', 'cube', 2 );
insert into bricks values ( 5, 'red', 'pyramid', 3 );
insert into bricks values ( 6, 'green', 'pyramid', 1 );
```

The group by clause splits rows into groups of the same value. For example, the following get the number of rows and total weight for each colour:

```
select colour, count(*), sum ( weight )
from bricks
group by colour;
```

You can carve up the input to an analytic function like this with the partition by clause. The following returns the total weight and count of rows of each colour. It includes all the rows:

Partition By in Analytical function is the same as **group by** in aggregate. Just need to focus on **partition by** column, as that value changes , we have to start again. This is the whole funda.

```
select B.*, count(*) over(partition by colour) bricks_per_colour,
count(*) over(partition by shape) bricks_per_shape,
sum(weight) over (partition by colour) weight_per_colour,
sum(weight) over (partition by shape) weight_per_shape
From Bricks B;
```

```
select B.*, count(*) over(partition by colour) bricks_per_colour,
count(*) over(partition by shape) shape_per_colour,
sum(weight) over(partition by colour) weight_per_colour,
sum(weight) over(partition by shape) weight_per_shape
from bricks B
```

BRICK_ID	COLOUR	SHAPE	WEIGHT	BRICKS_PER_COLOUR	SHAPE_PER_COLOUR	WEIGHT_PER_COLOUR	WEIGHT_PER_SHAPE
1	blue	cube	1	2	3	3	4
2	blue	pyramid	2	2	3	3	6
6	green	pyramid	1	1	3	1	6
5	red	pyramid	3	3	3	6	6
3	red	cube	1	3	3	6	4
4	red	cube	2	3	3	6	4

6 rows selected.

See the above query. Line #1, count(*) over(partition by colour) bricks_per_colour,

What does this mean???? I want a total count of bricks of each colour. each means **group by**, lets call it **partition by**. So, here it creates a boundary on column **colour**, once **colour** changes , we have to start again. <what starts again, count start again.>

See the above query. Line #2, count(*) over(partition by shape) bricks_per_shape,

What does this mean???? I want a total count of bricks of each shape. each means **group by**, lets call it **partition by**. So, here it creates a boundary on column **shape**, once **shape** changes , we have to start again. <what starts again, count start again.>

See the above query. Line #3, sum(weight) over (partition by colour) weight_per_colour ,

What does this mean???? I want a total weight of bricks of each colour. each means **group by**, lets call it **partition by**. So, here it creates a boundary on column **colour**, once **colour** changes , we have to start again. <what starts again, sum of weight start again.>

1 + 2 = 3

BRICK_ID	COLOUR	SHAPE	WEIGHT	BRICKS_PER_COLOR	BRICKS_PER_SHAPE	WEIGHT_PER_COLOR	WEIGHT_PER_SHAPE
1	blue	cube	1	2	3	3	4
2	blue	pyramid	2	2	4	3	8
7	green	pyramid	2	2	4	3	8
6	green	pyramid	1	2	4	3	8
5	red	pyramid	3	3	4	6	8
4	red	cube	2	3	3	6	4
3	red	cube	1	3	3	6	4

Download CSV
7 rows selected.

3 + 2 + 1 = 6

2 + 1 = 3

count total 3 for cube

BRICK_ID	COLOUR	SHAPE	WEIGHT	BRICKS_PER_COLOR	BRICKS_PER_SHAPE	WEIGHT_PER_COLOR	WEIGHT_PER_SHAPE
1	blue	cube	1	2	3 ✓	3	4
2	blue	pyramid	2	2	4 ✓	3	8
7	green	pyramid	2	2	4 ✓	3	8
6	green	pyramid	1	2	4 ✓	3	8
5	red	pyramid	3	3	4 ✓	6	8
4	red	cube	2	3	3 ✓	6	4
3	red	cube	1	3	3 ✓	6	4

Download CSV
7 rows selected.

4 records for Pyramid

2 Records of green

BRICK_ID	COLOUR	SHAPE	WEIGHT	BRICKS_PER_COLOR	BRICKS_PER_SHAPE	WEIGHT_PER_COLOR	WEIGHT_PER_SHAPE
1	blue	cube	1	2	3	3	4
2	blue	pyramid	2	2	4	3	8
7	green	pyramid	2	2	4	3	8
6	green	pyramid	1	2	4	3	8
5	red	pyramid	3	3	4	6	8
4	red	cube	2	3	3	6	4
3	red	cube	1	3	3	6	4

Download CSV
7 rows selected.

2 Records for Blue

3 records of Red

```
select H.*, max(salary) over(partition by department_id) max_sal_per_dept from hr.employees H
where department_id in (60,90,100)
```

SQL Worksheet

1 select H.*, max(salary) over(partition by department_id) max_sal_per_dept from hr.employees H
2 where department_id in (60,90,100)

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	MAX_SAL_PER_DEPT
103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-06	IT_PROG	9000	-	102	60	9000
104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-07	IT_PROG	6000	-	103	60	9000
105	David	Austin	DAUSTIN	590.423.4569	25-JUN-05	IT_PROG	4800	-	103	60	9000
106	Valli	Pataballa	VPATABAL	590.423.4560	05-FEB-06	IT_PROG	4800	-	103	60	9000
107	Diana	Lorentz	DLORENTZ	590.423.5567	07-FEB-07	IT_PROG	4200	-	103	60	9000
100	Steven	King	SKING	515.123.4567	17-JUN-03	AD_PRES	24000	-	-	90	24000

Suppose, lets want to know the details of employees, who is taking max sal from each department. Just find the max salary from each department and preserve my input data as it is and after that compare where salary = max salary

```
with n1 as (
select H.employee_id, H.first_name, H.last_name, H.department_id, H.salary, max(salary) over(partition by department_id) maxsal
from hr.employees H where department_id in (60,90,100) ) select * from n1 where SALARY = MAXSAL
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	DEPARTMENT_ID	SALARY	MAXSAL
103	Alexander	Hunold	60	9000	9000
100	Steven	King	90	24000	24000
108	Nancy	Greenberg	100	12008	12008

3 rows selected.